ETH zürich

DINFK



Gaussian Processes

Balázs Szekér & Maximilian Seeliger

08.10.2026

Agenda

1) Lecture Recap
   a) Linear Regression
   b) Bayesian Linear Regression
   c) Gaussian Processes
      i. Introduction
      ii. Kernels
      iii. Learning and Inference
      iv. Sampling
2) Old Exam Questions

ETH zürich

Lecture Recap: From Linear Regression to Gaussian Processes

ETH zürich

Linear Regression

Problem Setting

- Given: Dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ where $y \in \mathbb{R}$ and $\mathbf{x} \in \mathbb{R}^d$
- Assumption: Linear dependence of input and output variables: $y_i = \mathbf{w}^T \mathbf{x}_i + \epsilon_i$ where $\epsilon_i \sim \mathcal{N}(0, \sigma_n^2)$
- Goal: Find the «best» weight vector $\mathbf{w}$ such that the function $f(\mathbf{x}; \mathbf{w}) = \mathbf{w}^T \mathbf{x}$ «best» fits the data
- In vectorized notation, define $\mathbf{y} := [y_1, \dots, y_n]^T$, $\mathbf{f} := [f_1, \dots, f_n]^T$, and

$$\boldsymbol{X} \doteq \begin{bmatrix} \boldsymbol{x}_1^\top \\ \vdots \\ \boldsymbol{x}_n^\top \end{bmatrix} \in \mathbb{R}^{n \times d},$$

- Then, we try to find the weight vector $\mathbf{w}$ that satisfies $\mathbf{y} = \mathbf{f} + \boldsymbol{\epsilon} = \mathbf{X}\mathbf{w} + \boldsymbol{\epsilon}$ the «best»

Ordinary Least Squares

One of the most naïve estimate of the «best» weight vector $\mathbf{w}$ is the least squares estimator:

$$\hat{\boldsymbol{w}}_{\mathrm{ls}} \doteq \underset{\boldsymbol{w}\in\mathbb{R}^d}{\arg\min} \sum_{i=1}^{n} (y_i - \boldsymbol{w}^\top \boldsymbol{x}_i)^2 = \underset{\boldsymbol{w}\in\mathbb{R}^d}{\arg\min} \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}\|_2^2,$$

Ridge Regression

A slightly more sophisticated estimator can be found via ridge regression:

$$\hat{\boldsymbol{w}}_{\text{ridge}} \doteq \underset{\boldsymbol{w} \in \mathbb{R}^d}{\arg\min} \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}\|_2^2 + \lambda \|\boldsymbol{w}\|_2^2$$

OLS and Ridge Regression from a Bayesian Perspective

- Shared Setting: $y_i = \mathbf{w}^T \mathbf{x}_i + \epsilon_i$ where $\epsilon_i \overset{iid}{\sim} \mathcal{N}(0, \sigma_n^2)$
- Differences:

$p(w \mid x, y) = \frac{1}{Z} p(y \mid x, w)\, p(w)$

$\mathcal{U}$-prior $\Rightarrow p(w) \perp w$

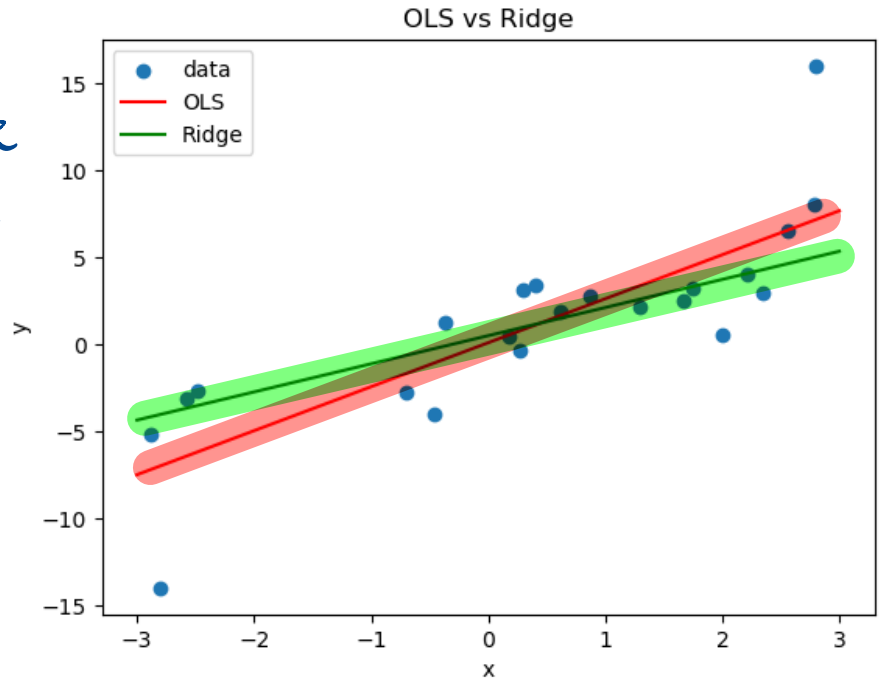
$p(w) = \text{const}$

	OLS	Ridge
Prior (on weight vector)	None (uniform prior: $p(w) \propto 1$)	$\mathbf{w} \sim \mathcal{N}\left(\mathbf{0}, \sigma_p^2 \mathbb{1}\right)$
Estimate	MLE: $\text{argmax}_{\mathbf{w} \in \mathbb{R}^d} \log\left(p(\mathbf{y} \mid \mathbf{X}, \mathbf{w})\right)$	MAP: $\text{argmax}_{\mathbf{w} \in \mathbb{R}^d} \log\left(p(\mathbf{y} \mid \mathbf{X}, \mathbf{w})\right) + \log\left(p(\mathbf{w})\right)$
Objective	$\text{argmin}_{\mathbf{w} \in \mathbb{R}^d} \lvert\lvert \mathbf{y} - \mathbf{X}\mathbf{w} \rvert\rvert^2$	$\text{argmin}_{\mathbf{w} \in \mathbb{R}^d} \lvert\lvert \mathbf{y} - \mathbf{X}\mathbf{w} \rvert\rvert^2 + \lambda \lvert\lvert \mathbf{w} \rvert\rvert^2$ where $\lambda = \sigma_n^2 / \sigma_p^2$
Solution	$\mathbf{w}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$	$\mathbf{w}^* = (\mathbf{X}^T\mathbf{X} + \lambda \mathbb{1})^{-1}\mathbf{X}^T\mathbf{y}$

Downsides of OLS and Ridge Regression

- OLS and Ridge Regression only yield a point estimate of the posterior weight vector $\mathbf{w}$, let's call it $\mathbf{w}^*$
- Then, at inference, we can also only make point predictions:
  - For a new point $\mathbf{x}$, we predict it's corresponding function value by calculating $f(\mathbf{x}) = (\mathbf{w}^*)^T\mathbf{x}$
- So, both, OLS and Ridge Regression, are missing any information about the uncertainty of the prediction!

- you can "add" the aleatoric uncertainty σ_n^2



$$y_i = f_i + \varepsilon_i$$
$$= (\underline{w}^*)^T \cdot \underline{x}_i + \varepsilon_i$$

$$\varepsilon_i \sim \mathcal{N}(0, \sigma_n^2)$$

$$y_i \sim \mathcal{N}\left((\underline{w}^*)^T \cdot \underline{x}_i, \sigma_n^2\right)$$

ETH zürich

Bayesian Linear Regression (BLR)

Bayesian Linear Regression

- Goal: go from point estimates to distributions
- Given: Dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$ where $y \in \mathbb{R}$ and $\mathbf{x} \in \mathbb{R}^d$

Bayesian Linear Regression

- Goal: go from point estimates to distributions
- Given: Dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ where $y \in \mathbb{R}$ and $\mathbf{x} \in \mathbb{R}^d$
- Assumptions:
  - Linear dependence of input and output variables: $y_i = \mathbf{w}^T \mathbf{x}_i + \epsilon_i$ where $\epsilon_i \sim \mathcal{N}(0, \sigma_n^2)$
    - Hence, we can write: $p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) = \mathcal{N}(\mathbf{y}; \mathbf{X}\mathbf{w}, \sigma_n^2 \mathbb{1})$
  - Zero-centered gaussian prior on the weights: $p(\mathbf{w}) = \mathcal{N}\left(\mathbf{0}, \sigma_p^2 \mathbb{1}\right)$

Bayesian Linear Regression

- Goal: go from point estimates to distributions
- Given: Dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ where $y \in \mathbb{R}$ and $\mathbf{x} \in \mathbb{R}^d$
- Assumptions:
  - Linear dependence of input and output variables: $y_i = \mathbf{w}^T \mathbf{x}_i + \epsilon_i$ where $\epsilon_i \sim \mathcal{N}(0, \sigma_n^2)$
    - Hence, we can write: $p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) = \mathcal{N}(\mathbf{y}; \mathbf{X}\mathbf{w}, \sigma_n^2 \mathbb{1})$
  - Zero-centered gaussian prior on the weights: $p(\mathbf{w}) = \mathcal{N}\left(\mathbf{0}, \sigma_p^2 \mathbb{1}\right)$
- We can calculate the posterior distribution over the weights $\mathbf{w}$ with Bayes Rule:
  - $p(\mathbf{w} \mid \mathbf{X}, \mathbf{y}) = \frac{1}{Z} p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) p(\mathbf{w}) = \mathcal{N}(\mathbf{w}; \boldsymbol{\mu}, \Sigma)$
  - $\boldsymbol{\mu} = \left(\mathbf{X}^T \mathbf{X} + \frac{\sigma_n^2}{\sigma_p^2} \mathbb{1}\right)^{-1} \mathbf{X}^T \mathbf{y}$ ← $\hat{w}_{MAP} = \hat{w}_{Ridge}$
  - $\Sigma = \sigma_n^2 \left(\mathbf{X}^T \mathbf{X} + \frac{\sigma_n^2}{\sigma_p^2} \mathbb{1}\right)^{-1}$

Bayesian Linear Regression

Given the posterior distribution $p(\mathbf{w} \mid \mathbf{X}, \mathbf{y}) = \mathcal{N}(\mathbf{w}; \boldsymbol{\mu}, \Sigma)$, we can define the predictive distribution at some point $\mathbf{x}^*$ (since $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x}^*$ and $y_i = f(\mathbf{x}_i) + \epsilon_i$):

$$w \sim \mathcal{N}(\mu_w, \Sigma_1)$$

$$f^* = w^T \cdot x^* \sim \mathcal{N}(\mu_w^T \cdot x^*, (x^*)^T \Sigma_1 (x^*))$$

$$y^* = f^* + \varepsilon \sim \mathcal{N}(\mu_w^T x^*, (x^*)^T \Sigma x^* + \sigma_n^2)$$

Bayesian Linear Regression vs Ridge Regression

Bayesian Linear Regression:

- Predict by averaging all $\mathbf{w}$ according to the posterior:
- $p\left(y^* \mid \mathbf{x}^*, \mathbf{x}_{1:n}, y_{1:n}\right)$
$$= \int p\left(y^* \mid \mathbf{x}^*, \mathbf{w}\right) p(\mathbf{w} \mid \mathbf{x}_{1:n}, y_{1:n}) d\mathbf{w}$$
$$= \mathcal{N}(y^*; \boldsymbol{\mu}^T \mathbf{x}^*, (\mathbf{x}^*)^T \Sigma \mathbf{x}^* + \sigma_n^2)$$

Ridge Regression:

- Collapse all uncertainty about the posterior distribution onto the mode!
- $\hat{\mathbf{w}} = \operatorname{argmax}_{\mathbf{w} \in \mathbb{R}^d} p(\mathbf{w} \mid \mathbf{x}_{1:n}, y_{1:n})$
- $p(y^* \mid \mathbf{x}^*, \mathbf{w}) = \mathcal{N}(y^*; \hat{\mathbf{w}}^T \mathbf{x}^*, \sigma_n^2)$

Interpretation: Ridge Regression drops the epistemic uncertainty!

ETH zürich

Gaussian Processes Introduction

From Linear to Non-Linear Features

- So far, we have used $\mathbf{X}$ as the input to our prediction model
- We can also apply a non-linear transformation $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^e$ to the features $\mathbf{x}_i$
- We can then the define the feature matrix $\mathbf{\Phi} \in \mathbb{R}^{n \times e}$
- With this update, the functions we can model take the form: $\mathbf{f} = \mathbf{\Phi w}$

Linear in w

$$\Phi = \begin{bmatrix} \phi^T(x_1) \\ \vdots \\ \phi^T(x_n) \end{bmatrix} \in \mathbb{R}^{n \times e}$$

From Weight Space to Function Space

- Instead of considering a prior over the weights $\mathbf{w} \sim \mathcal{N}\left(\mathbf{0}, \sigma_p^2 \mathbb{1}\right)$, we can consider a prior over functions!

From Weight Space to Function Space

- Instead of considering a prior over the weights $\mathbf{w} \sim \mathcal{N}\left(\mathbf{0}, \sigma_p^2 \mathbb{1}\right)$, we can consider a prior over functions!
- If $\mathbf{f} = \mathbf{\Phi w}$, and $\mathbf{w} \sim \mathcal{N}\left(\mathbf{0}, \sigma_p^2 \mathbb{1}\right)$, then it holds that
  - $\mathbf{f} \mid \mathbf{X} \sim \mathcal{N}(\mathbf{\Phi}\mathbb{E}[\mathbf{w}], \mathbf{\Phi}\mathrm{Var}[\mathbf{w}]\mathbf{\Phi}^T) = \mathcal{N}\left(\mathbf{0}, \sigma_p^2 \mathbf{\Phi}\mathbf{\Phi}^T\right)$

From Weight Space to Function Space

- Instead of considering a prior over the weights $\mathbf{w} \sim \mathcal{N}\left(\mathbf{0}, \sigma_p^2 \mathbb{1}\right)$, we can consider a prior over functions!
- If $\mathbf{f} = \mathbf{\Phi w}$, and $\mathbf{w} \sim \mathcal{N}\left(\mathbf{0}, \sigma_p^2 \mathbb{1}\right)$, then it holds that
  - $\mathbf{f} \mid \mathbf{X} \sim \mathcal{N}(\mathbf{\Phi}\mathbb{E}[\mathbf{w}], \mathbf{\Phi}\mathrm{Var}[\mathbf{w}]\mathbf{\Phi}^T) = \mathcal{N}\left(\mathbf{0}, \sigma_p^2 \mathbf{\Phi\Phi}^T\right) = \mathcal{N}(0, K)$
- Define the kernel matrix $\mathbf{K} = \sigma_p^2 \mathbf{\Phi\Phi}^T \in \mathbb{R}^{n \times n}$
  - It holds that $\mathbf{K}(i,j) = \sigma_p^2 \boldsymbol{\phi}(\mathbf{x}_i)^T \boldsymbol{\phi}\left(\mathbf{x}_j\right)$

From Weight Space to Function Space

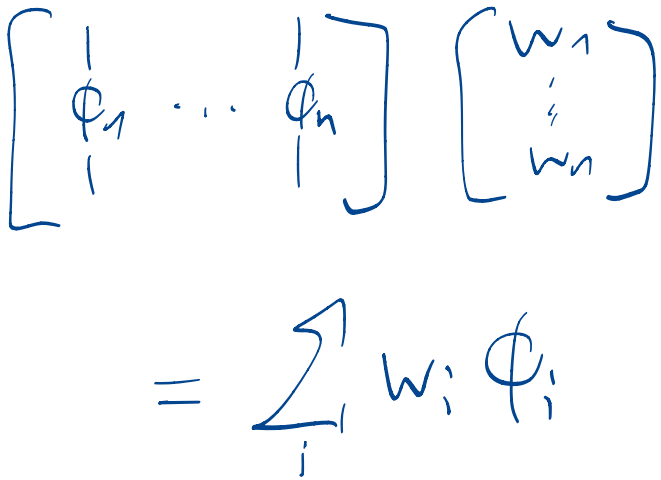
- Instead of considering a prior over the weights $\mathbf{w} \sim \mathcal{N}\left(\mathbf{0}, \sigma_p^2 \mathbb{1}\right)$, we can consider a prior over functions!
- If $\mathbf{f} = \boldsymbol{\Phi}\mathbf{w}$, and $\mathbf{w} \sim \mathcal{N}\left(\mathbf{0}, \sigma_p^2 \mathbb{1}\right)$, then it holds that
  - $\mathbf{f} \mid \mathbf{X} \sim \mathcal{N}(\boldsymbol{\Phi}\mathbb{E}[\mathbf{w}], \boldsymbol{\Phi}\mathrm{Var}[\mathbf{w}]\boldsymbol{\Phi}^T) = \mathcal{N}\left(\mathbf{0}, \sigma_p^2 \boldsymbol{\Phi}\boldsymbol{\Phi}^T\right)$
- Define the kernel matrix $\mathbf{K} = \sigma_p^2 \boldsymbol{\Phi}\boldsymbol{\Phi}^T \in \mathbb{R}^{n \times n}$
  - It holds that $\mathbf{K}(i,j) = \sigma_p^2 \boldsymbol{\phi}(\mathbf{x}_i)^T \boldsymbol{\phi}\left(\mathbf{x}_j\right)$
- Define the kernel function as:
  - $\mathrm{k}(\mathbf{x}, \mathbf{x}') = \sigma_p^2 \boldsymbol{\phi}(\mathbf{x})^T \boldsymbol{\phi}(\mathbf{x}')$

From Weight Space to Function Space

- Instead of considering a prior over the weights $\mathbf{w} \sim \mathcal{N}\left(\mathbf{0}, \sigma_p^2 \mathbb{1}\right)$, we can consider a prior over functions!
- If $\mathbf{f} = \mathbf{\Phi w}$, and $\mathbf{w} \sim \mathcal{N}\left(\mathbf{0}, \sigma_p^2 \mathbb{1}\right)$, then it holds that
  - $\mathbf{f} \mid \mathbf{X} \sim \mathcal{N}(\mathbf{\Phi}\mathbb{E}[\mathbf{w}], \mathbf{\Phi}\mathrm{Var}[\mathbf{w}]\mathbf{\Phi}^T) = \mathcal{N}\left(\mathbf{0}, \sigma_p^2 \mathbf{\Phi\Phi}^T\right)$
- Define the kernel matrix $\mathbf{K} = \sigma_p^2 \mathbf{\Phi\Phi}^T \in \mathbb{R}^{n \times n}$
  - It holds that $\mathbf{K}(i,j) = \sigma_p^2 \boldsymbol{\phi}(\mathbf{x}_i)^T \boldsymbol{\phi}\left(\mathbf{x}_j\right)$
- Define the kernel function as:
  - $\mathrm{k}(\mathbf{x}, \mathbf{x}') = \sigma_p^2 \boldsymbol{\phi}(\mathbf{x})^T \boldsymbol{\phi}(\mathbf{x}')$
- The kernel function can be interpreted as the covariance of the function values:
  - $\mathrm{k}(\mathbf{x}, \mathbf{x}') = \mathrm{Cov}[f(\mathbf{x}), f(\mathbf{x}')]$ → for 2 input variables $x, x' \in \mathcal{X}$ how much do $f(x)$ & $f(x')$ correlate

From Weight Space to Function Space

- Instead of considering a prior over the weights $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \sigma_p^2 \mathbb{1})$, we can consider a prior over functions!
- If $\mathbf{f} = \mathbf{\Phi w}$, and $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \sigma_p^2 \mathbb{1})$, then it holds that
  - $\mathbf{f} \mid \mathbf{X} \sim \mathcal{N}(\mathbf{\Phi}\mathbb{E}[\mathbf{w}], \mathbf{\Phi}\mathrm{Var}[\mathbf{w}]\mathbf{\Phi}^T) = \mathcal{N}(\mathbf{0}, \sigma_p^2 \mathbf{\Phi\Phi}^T)$
- Define the kernel matrix $\mathbf{K} = \sigma_p^2 \mathbf{\Phi\Phi}^T \in \mathbb{R}^{n \times n}$
  - It holds that $\mathbf{K}(i,j) = \sigma_p^2 \boldsymbol{\phi}(\mathbf{x}_i)^T \boldsymbol{\phi}(\mathbf{x}_j)$
- Define the kernel function as:
  - $\mathrm{k}(\mathbf{x}, \mathbf{x}') = \sigma_p^2 \boldsymbol{\phi}(\mathbf{x})^T \boldsymbol{\phi}(\mathbf{x}')$
- The kernel function can be interpreted as the covariance of the function values:
  - $\mathrm{k}(\mathbf{x}, \mathbf{x}') = \mathrm{Cov}[f(\mathbf{x}), f(\mathbf{x}')]$
- Hence, the kernel matrix can be interpreted as a covariance matrix.
  - $\mathbf{K} = \begin{bmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & \dots & k(\mathbf{x}_1, \mathbf{x}_n) \\ \vdots & \ddots & \vdots \\ k(\mathbf{x}_n, \mathbf{x}_1) & \dots & k(\mathbf{x}_n, \mathbf{x}_n) \end{bmatrix}$



Kernel Trick

Why is the Kernel Trick useful?

- For large dimensions e it becomes infeasible to compute the feature vector $\boldsymbol{\phi}(\mathbf{x})$ and the dot product $\boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{x}')$
- Instead of computing the dot product explicitely, we want to find a (simple) function $k: \mathcal{X}\times\mathcal{X} \rightarrow \mathbb{R}$ such that $k(\mathbf{x}, \mathbf{x}') = \boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{x}')$

Kernel Trick

Why is the Kernel Trick useful?

- For large dimensions e it becomes infeasible to compute the feature vector $\boldsymbol{\phi}(\mathbf{x})$ and the dot product $\boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{x}')$
- Instead of computing the dot product explicitely, we want to find a (simple) function $k: \mathcal{X}\times\mathcal{X} \to \mathbb{R}$ such that $k(\mathbf{x}, \mathbf{x}') = \boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{x}')$

Remarks:

- When working with kernels, the feature space is implicit in the choice of the kernel.
- The choice of kernel implicitly determines the class of functions that $\mathbf{f}$ is sampled from.
- This choice encodes some prior belief.

Kernel Trick

Why is the Kernel Trick useful?

- For large dimensions e it becomes infeasible to compute the feature vector $\boldsymbol{\phi}(\mathbf{x})$ and the dot product $\boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{x}')$
- Instead of computing the dot product explicitely, we want to find a (simple) function $k: \mathcal{X}\times\mathcal{X} \to \mathbb{R}$ such that $k(\mathbf{x}, \mathbf{x}') = \boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{x}')$

Remarks:

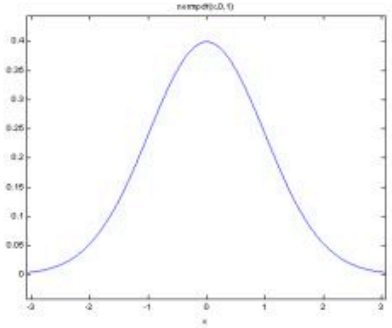
- When working with kernels, the feature space is implicit in the choice of the kernel.
- The choice of kernel implicitly determines the class of functions that $\mathbf{f}$ is sampled from.
- This choice encodes some prior belief.

More on kernels soon!

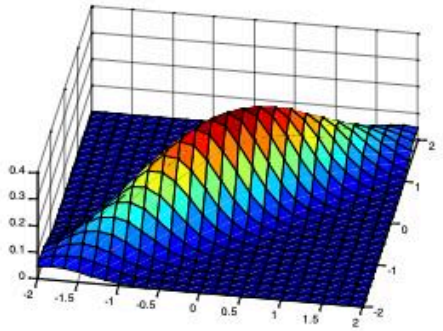
Formal Definition of Gaussian Processes

$f_i = f(x_i)$ $\int \mathcal{N}(f_1, \dots, f_n)\, df_i = \mathcal{N}(f_1, \dots, f_{i-1}, f_{i+1}, \dots, f_n)$

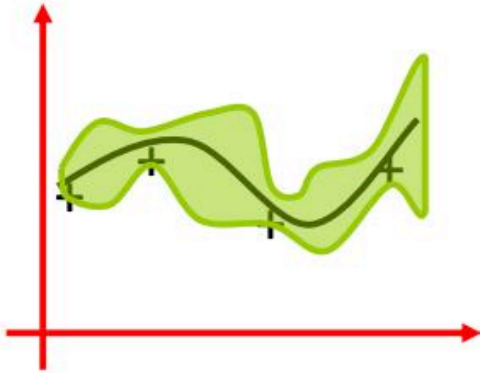
A Gaussian Process is an infinite set of random variables such that any finite number of them are jointly Gaussian and such that they are consistent under marginalization.



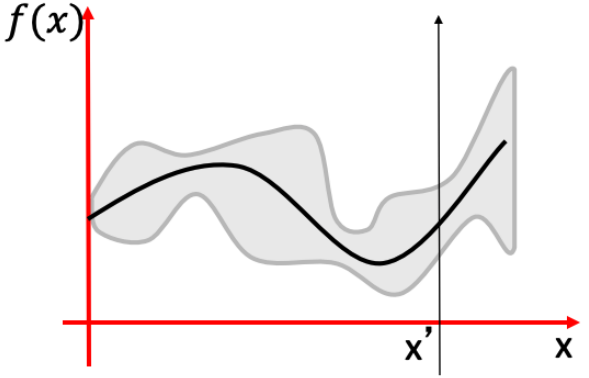
Normal dist.
(1-D Gaussian)

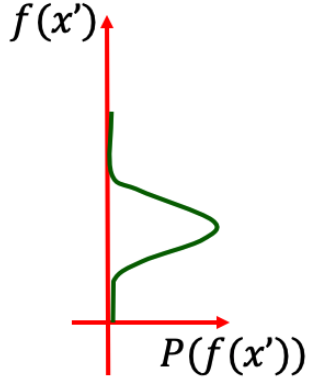


Multivariate normal
(n-D Gaussian)



Gaussian process
(∞-D Gaussian)





$x_1, \dots, x_n \rightarrow f(x_1), \dots, f(x_n)$

$\rightarrow \begin{bmatrix} f_1 \\ \vdots \\ f_n \end{bmatrix} \sim \mathcal{N}(0, K)$

Characterization of Gaussian Processes

A Gaussian Process is characterized by a mean function $\mu: \mathcal{X} \to \mathbb{R}$ and a covariance function (or kernel function) $k: \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ such that for any set of points $A = \{\mathbf{x}_1, \ldots, \mathbf{x}_m\} \subseteq \mathcal{X}$ we have

$$\mathbf{f}_A = \left[f_{\mathbf{x}_1}, \ldots, f_{\mathbf{x}_m}\right]^T \sim \mathcal{N}(\boldsymbol{\mu}_A, \mathbf{K}_{AA})$$

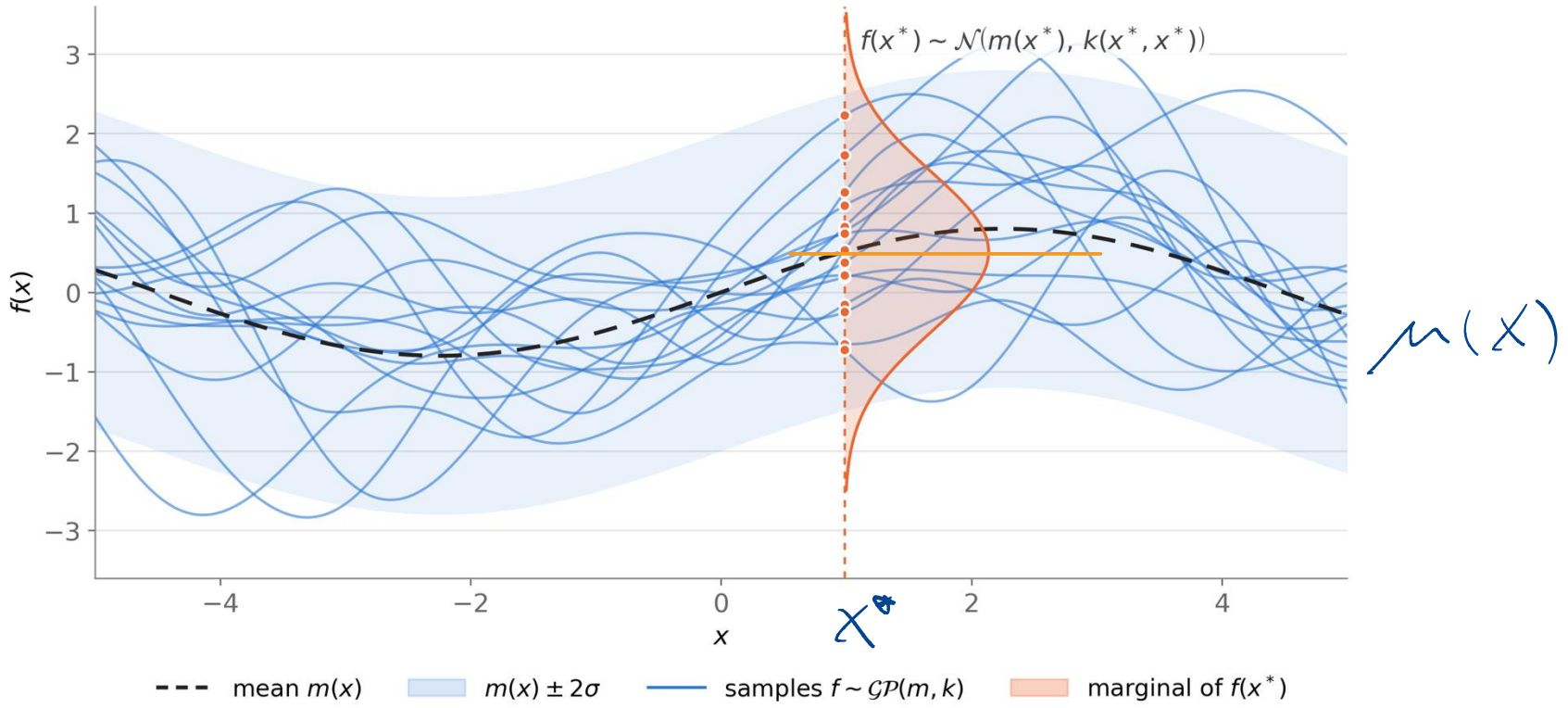
where

$$\boldsymbol{\mu}_A \doteq \begin{bmatrix} \mu(\boldsymbol{x}_1) \\ \vdots \\ \mu(\boldsymbol{x}_m) \end{bmatrix}, \quad \boldsymbol{K}_{AA} \doteq \begin{bmatrix} k(\boldsymbol{x}_1, \boldsymbol{x}_1) & \cdots & k(\boldsymbol{x}_1, \boldsymbol{x}_m) \\ \vdots & \ddots & \vdots \\ k(\boldsymbol{x}_m, \boldsymbol{x}_1) & \cdots & k(\boldsymbol{x}_m, \boldsymbol{x}_m) \end{bmatrix}$$

We write $f \sim \mathcal{GP}(\mu, k)$

Intuition behind Gaussian Processes

A Gaussian Process can be interpreted as a normal distribution over functions



(This image was generated by Claude)

ETH zürich

Gaussian Processes
Kernels

Formal Definition of a Kernel Function

A kernel function $k: \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ satisfies

- Symmetry: $k(\mathbf{x}, \mathbf{x}') = k(\mathbf{x}', \mathbf{x}) \quad \forall \mathbf{x}, \mathbf{x}' \in \mathcal{X}$
- Positive Semi-Definiteness: $\mathbf{K}_{AA}$ is a positive semi-definite matrix for any $A \subseteq \mathcal{X}$

$$\forall v \in \mathbb{R}^n : \quad v^T K_{AA} v \geq 0$$

Intuition behind Kerels

- The kernel function evaluated at locations $\mathbf{x}$ and $\mathbf{x}$' describes how $f(\mathbf{x})$ and $f(\mathbf{x}')$ are related
- Mathematically, one can write: $k(\mathbf{x}, \mathbf{x}') = \mathrm{Cov}[f(\mathbf{x}), f(\mathbf{x}')]$
- Hence, under the assumptions that we are modelling smooth functions, if $\mathbf{x}$ and $\mathbf{x}$' are «close», then $f(\mathbf{x})$ and $f(\mathbf{x}')$ are usually positively correlated.

Addendum Kernel Trick

- We introduced kernels to get rid of the dot product $\boldsymbol{\phi}(\mathbf{x})^T \boldsymbol{\phi}(\mathbf{x}')$ in the covariance / kernel matrix

Addendum Kernel Trick

- We introduced kernels to get rid of the dot product $\boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{x}')$ in the covariance / kernel matrix
- Our goal is to find a (simple) function $k: \mathcal{X}\times\mathcal{X} \to \mathbb{R}$ such that $k(\mathbf{x}, \mathbf{x}') = \boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{x}')$

Addendum Kernel Trick

- We introduced kernels to get rid of the dot product $\boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{x}')$ in the covariance / kernel matrix
- Our goal is to find a (simple) function $k: \mathcal{X}\times\mathcal{X} \to \mathbb{R}$ such that $k(\mathbf{x}, \mathbf{x}') = \boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{x}')$
- Our options are:

Addendum Kernel Trick

- We introduced kernels to get rid of the dot product $\boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{x}')$ in the covariance / kernel matrix
- Our goal is to find a (simple) function $k: \mathcal{X}\times\mathcal{X} \to \mathbb{R}$ such that $k(\mathbf{x}, \mathbf{x}') = \boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{x}')$
- Our options are:
  - We may be able to find an easier to compute expression equivalent to $\boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{x}')$
    - E.g. for the polynomial feature transformation $\boldsymbol{\phi}(\mathbf{x})$ up to degree m, it can be shown that up to constant factors: $\boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{x}') = (1 + \mathbf{x}^T\mathbf{x}')^m$

Addendum Kernel Trick

- We introduced kernels to get rid of the dot product $\boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{x}')$ in the covariance / kernel matrix
- Our goal is to find a (simple) function $k: \mathcal{X}\times\mathcal{X} \to \mathbb{R}$ such that $k(\mathbf{x}, \mathbf{x}') = \boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{x}')$
- Our options are:
  - We may be able to find an easier to compute expression equivalent to $\boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{x}')$
    - E.g. for the polynomial feature transformation $\boldsymbol{\phi}(\mathbf{x})$ up to degree m, it can be shown that up to constant factors: $\boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{x}') = (1 + \mathbf{x}^T\mathbf{x}')^m$
  - If this is not possible, we could approximate the inner product by an easier to compute expression

Addendum Kernel Trick

- We introduced kernels to get rid of the dot product $\boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{x}')$ in the covariance / kernel matrix
- Our goal is to find a (simple) function $k: \mathcal{X}\times\mathcal{X} \to \mathbb{R}$ such that $k(\mathbf{x}, \mathbf{x}') = \boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{x}')$
- Our options are:
  - We may be able to find an easier to compute expression equivalent to $\boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{x}')$
    - E.g. for the polynomial feature transformation $\boldsymbol{\phi}(\mathbf{x})$ up to degree m, it can be shown that up to constant factors: $\boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{x}') = (1 + \mathbf{x}^T\mathbf{x}')^m$
  - If this is not possible, we could approximate the inner product by an easier to compute expression
  - We may decide not to care very much about the exact feature transformation and simply experiment with different kernels that induce *some* feature space.

Addendum Kernel Trick

- We introduced kernels to get rid of the dot product $\boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{x}')$ in the covariance / kernel matrix
- Our goal is to find a (simple) function $k: \mathcal{X}\times\mathcal{X} \to \mathbb{R}$ such that $k(\mathbf{x}, \mathbf{x}') = \boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{x}')$
- Our options are:
  - We may be able to find an easier to compute expression equivalent to $\boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{x}')$
    - E.g. for the polynomial feature transformation $\boldsymbol{\phi}(\mathbf{x})$ up to degree m, it can be shown that up to constant factors: $\boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{x}') = (1 + \mathbf{x}^T\mathbf{x}')^m$
  - If this is not possible, we could approximate the inner product by an easier to compute expression
  - We may decide not to care very much about the exact feature transformation and simply experiment with different kernels that induce *some* feature space.
- The latter is what we do in Gaussian Processes

Common Kernels

- Linear Kernel
- Gaussian Kernel / Radial Basis Function (RBF) Kernel
- Laplace Kernel
- Matérn Kernel

Linear Kernel

- $k(\mathbf{x}, \mathbf{x}'; \boldsymbol{\phi}) = \boldsymbol{\phi}(\mathbf{x})^T \boldsymbol{\phi}(\mathbf{x})$
- where $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^e$ is a non-linear transformation (or the identity)

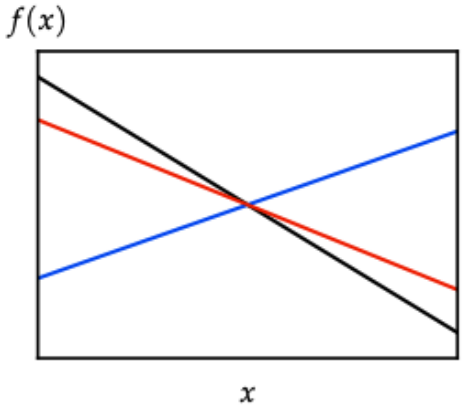


Figure 4.2: Functions sampled according to a Gaussian process with a linear kernel and $\phi = \mathrm{id}$.

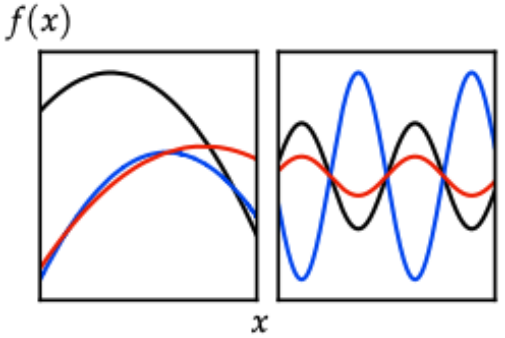


Figure 4.3: Functions sampled according to a Gaussian process with a linear kernel and $\boldsymbol{\phi}(x) = [1, x, x^2]$ (left) and $\phi(x) = \sin(x)$ (right).

Linear Kernel

- $k(\mathbf{x}, \mathbf{x}'; \boldsymbol{\phi}) = \boldsymbol{\phi}(\mathbf{x})^T \boldsymbol{\phi}(\mathbf{x}')$
- where $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^e$ is a non-linear transformation (or the identity)
- Remark:
  - A Gaussian Process with linear kernel is equivalent to Bayesian Linear Regression (see Homework)
  - Hence, BLR is a subclass / special case of GPs

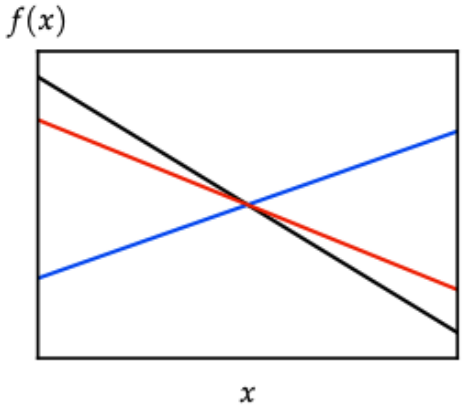


Figure 4.2: Functions sampled according to a Gaussian process with a linear kernel and $\phi = \text{id}$.

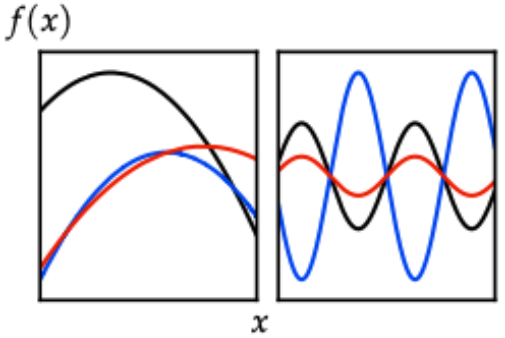
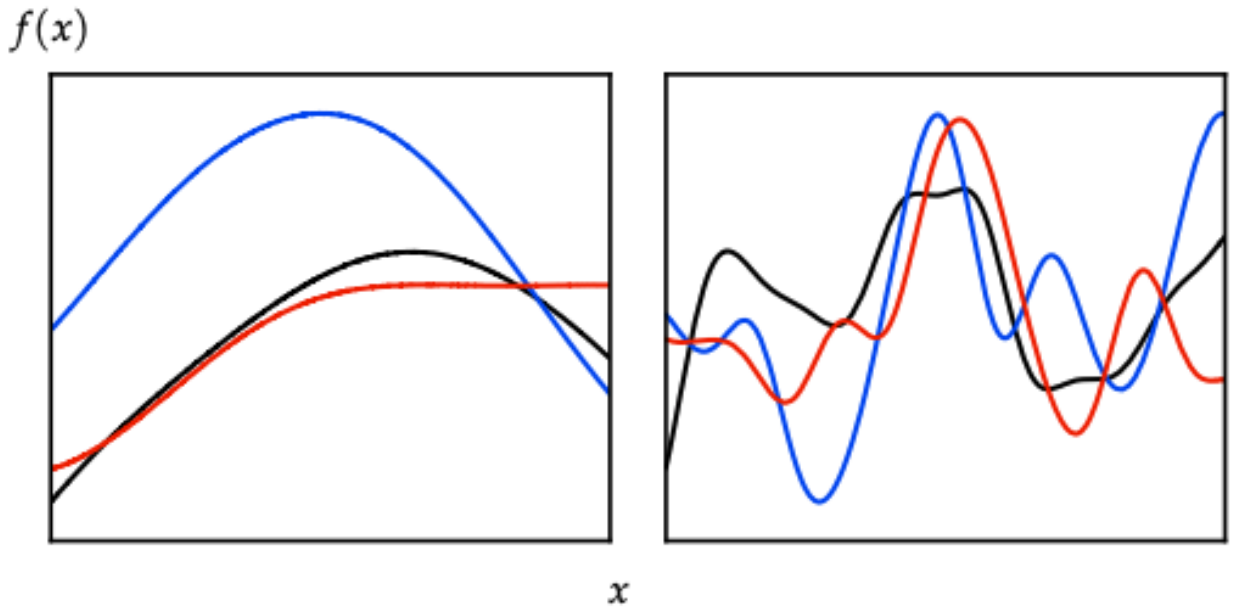


Figure 4.3: Functions sampled according to a Gaussian process with a linear kernel and $\boldsymbol{\phi}(x) = [1, x, x^2]$ (left) and $\phi(x) = \sin(x)$ (right).

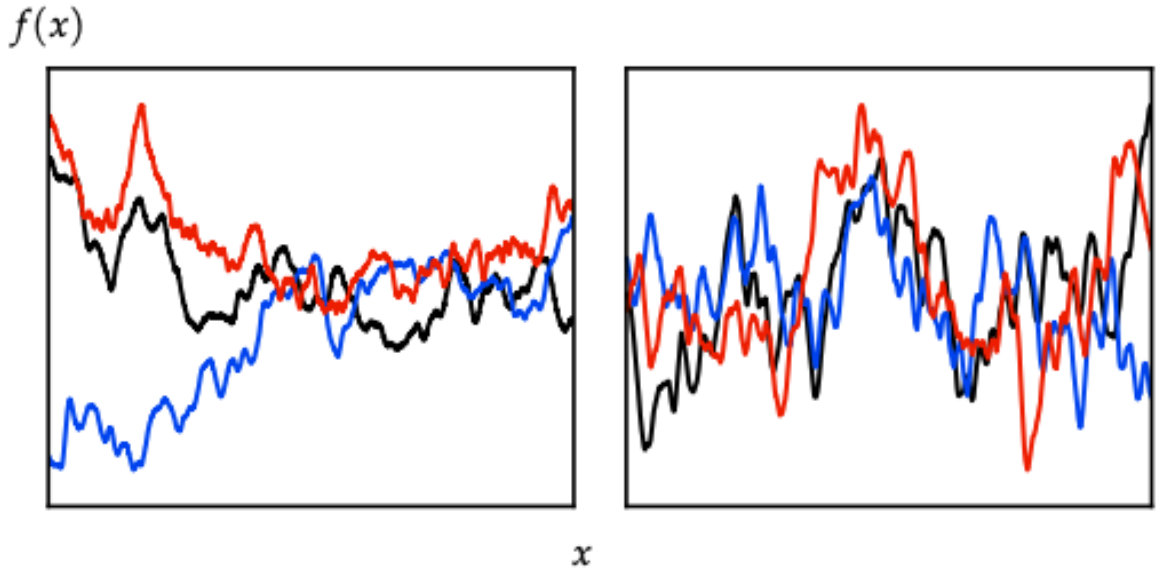
Gaussian / RBF Kernel

- $k(\mathbf{x}, \mathbf{x}'; h) = \exp\left(-\frac{||\mathbf{x}-\mathbf{x}'||_2^2}{2h^2}\right)$
- The parameter h is the length scale
  - The larger h, the smoother the resulting function
  - Larger h means longer-ranging correlation



Laplace Kernel

- $k(\mathbf{x}, \mathbf{x}'; h) = \exp\left(-\frac{||\mathbf{x}-\mathbf{x}'||_2}{h}\right)$
- The parameter h is the length scale
  - Larger h means longer-ranging correlation
- Samples from GPs with Laplace kernel are non-smooth as opposed to samples from GPs with Gaussian kernel.



Matérn Kernel

- $k(\mathbf{x}, \mathbf{x}'; \nu, h) = \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu}||\mathbf{x}-\mathbf{x}'||_2}{h} \right)^{\nu} K_{\nu} \left(\frac{\sqrt{2\nu}||\mathbf{x}-\mathbf{x}'||_2}{h} \right)$
- The parameter h is the length scale
  - Larger h means longer-ranging correlation
- Γ is the Gamma function
- K_{ν} is the modified Bessel function of the second kind
- ν is a parameter governing the smoothness of the sampled functions
  - For $\nu = 1/2$, the Matérn Kernel is equivalent to the Laplace kernel
  - For $\nu \to \infty$, the Matérn Kernel is equivalent to the Gaussian kernel

Composition Rules for Kernels

Given two kernels $k_1: \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ and $k_2: \mathcal{X} \times \mathcal{X} \to \mathbb{R}$, they can be composed to obtain a new kernel in the following ways:

- $k(\mathbf{x}, \mathbf{x}') = k_1(\mathbf{x}, \mathbf{x}') + k_2(\mathbf{x}, \mathbf{x}')$
- $k(\mathbf{x}, \mathbf{x}') = k_1(\mathbf{x}, \mathbf{x}') \cdot k_2(\mathbf{x}, \mathbf{x}')$ → $k_1 \cdot k_1 = k_1^2$ → k_1^m
- $k(\mathbf{x}, \mathbf{x}') = c \cdot k_1(\mathbf{x}, \mathbf{x}')$ for any $c \in \mathbb{R}_{>0}$
- $k(\mathbf{x}, \mathbf{x}') = f\big(k_1(\mathbf{x}, \mathbf{x}')\big)$ for any polynomial f with positive coefficients or $f = \exp$

Composition Rules for Kernels

Given two kernels $k_1: \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ and $k_2: \mathcal{X} \times \mathcal{X} \to \mathbb{R}$, they can be composed to obtain a new kernel in the following ways:

- $k(\mathbf{x}, \mathbf{x}') = k_1(\mathbf{x}, \mathbf{x}') + k_2(\mathbf{x}, \mathbf{x}')$
- $k(\mathbf{x}, \mathbf{x}') = k_1(\mathbf{x}, \mathbf{x}') \cdot k_2(\mathbf{x}, \mathbf{x}')$
- $k(\mathbf{x}, \mathbf{x}') = c \cdot k_1(\mathbf{x}, \mathbf{x}')$ for any $c \in \mathbb{R}_{>0}$
- $k(\mathbf{x}, \mathbf{x}') = f\left(k_1(\mathbf{x}, \mathbf{x}')\right)$ for any polynomial f with positive coefficients or $f = \exp$

Remarks:

- Addition of kernels can be thought of as an OR operation
- Multiplication of kernels can be thought of as an AND operation

Stationarity and Isotropy

A kernel $k: \mathbb{R}^d \times R^d \rightarrow \mathbb{R}$ is called

- Stationary (or shift-invariant) if there exists a function $\tilde{k}$ such that $k(\mathbf{x}, \mathbf{x}') = \tilde{k}(\mathbf{x} - \mathbf{x}')$
- Isotropic if there exists a function $\tilde{k}$ such that $k(\mathbf{x}, \mathbf{x}') = \tilde{k}(|| \mathbf{x} - \mathbf{x}' ||_2)$

Stationarity and Isotropy

A kernel k: $\mathbb{R}^d \times R^d \rightarrow \mathbb{R}$ is called

- Stationary (or shift-invariant) if there exists a function $\tilde{k}$ such that $k(\mathbf{x}, \mathbf{x}') = \tilde{k}(\mathbf{x} - \mathbf{x}')$
- Isotropic if there exists a function $\tilde{k}$ such that $k(\mathbf{x}, \mathbf{x}') = \tilde{k}(|| \mathbf{x} - \mathbf{x}' ||_2)$

Remark:

- isotropy implies stationarity

Stationarity and Isotropy

A kernel $k: \mathbb{R}^d \times R^d \rightarrow \mathbb{R}$ is called

- Stationary (or shift-invariant) if there exists a function $\tilde{k}$ such that $k(\mathbf{x}, \mathbf{x}') = \tilde{k}(\mathbf{x} - \mathbf{x}')$
- Isotropic if there exists a function $\tilde{k}$ such that $k(\mathbf{x}, \mathbf{x}') = \tilde{k}(|| \mathbf{x} - \mathbf{x}' ||_2)$

Remark:

- isotropy implies stationarity

$M \propto \mathbb{1}$

$\|\underline{x}-\underline{x}'\|_M^2 = (\underline{x}-\underline{x}')^T M (\underline{x}-\underline{x}')$

$= (\underline{x}-\underline{x}')^T c \mathbb{1} (\underline{x}-\underline{x}')$

$= c \cdot (\underline{x}-\underline{x}')^T (\underline{x}-\underline{x}')$

$= c \cdot \|\underline{x}-\underline{x}'\|_2^2$

$f(\|\underline{x}-\underline{x}'\|_2)$

Examples:

	stationary	isotropic
Linear Kernel	no	no
Gaussian Kernel	yes	yes
$k(\mathbf{x}, \mathbf{x}') = exp(-\|| \mathbf{x} - \mathbf{x}' ||_M^2)$ where $\mathbf{M}$ is positive semi-definite (When is it also isotropic?)	yes	no

Hyperparameter Optimization

- Will be covered in tomorrow's lecture!
- We will revisit it next week!

ETH zürich

Gaussian Processes
Learning and Inference

Learning and Inference

- Denote the set of observed points by $A = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$
- Given: prior $f \sim \mathcal{GP}(\mu, k)$ and the noisy observations $y_i = f(\mathbf{x}_i) + \epsilon_i$ with $\epsilon_i \sim \mathcal{N}(0, \sigma_n^2)$

Learning and Inference

- Denote the set of observed points by $A = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$
- Given: prior $f \sim \mathcal{GP}(\mu, k)$ and the noisy observations $y_i = f(\mathbf{x}_i) + \epsilon_i$ with $\epsilon_i \sim \mathcal{N}(0, \sigma_n^2)$
- The joint distribution of the observations $y_{1:n}$ and the noise free prediction f^* at a test point $\mathbf{x}^*$ can be written as:

$$\begin{bmatrix} y \\ f^* \end{bmatrix} \mid x^*, X \sim \mathcal{N}(\tilde{\mu}, \tilde{K})$$

$$\tilde{\mu} = \begin{bmatrix} \mu_A \\ \mu(x^*) \end{bmatrix}, \quad \tilde{K} = \begin{bmatrix} K_{AA} + \sigma_n^2 \mathbb{1} & K_{x^* A} \\ K_{x^* A}^T & k(x^*, x^*) \end{bmatrix}$$

$$K_{xA} = \begin{bmatrix} k(x, x_1) \\ \vdots \\ k(x, x_n) \end{bmatrix}$$

Learning and Inference

- Denote the set of observed points by $A = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$
- Given: prior $f \sim \mathcal{GP}(\mu, k)$ and the noisy observations $y_i = f(\mathbf{x}_i) + \epsilon_i$ with $\epsilon_i \sim \mathcal{N}(0, \sigma_n^2)$
- The joint distribution of the observations $y_{1:n}$ and the noise free prediction f^* at a test point $\mathbf{x}^*$ can be written as:

$$\begin{bmatrix} \boldsymbol{y} \\ f^\star \end{bmatrix} \mid \boldsymbol{x}^\star, \boldsymbol{X} \sim \mathcal{N}(\tilde{\boldsymbol{\mu}}, \tilde{\boldsymbol{K}}), \quad \text{where} \qquad (4.$$

$$\tilde{\boldsymbol{\mu}} \doteq \begin{bmatrix} \boldsymbol{\mu}_A \\ \mu(\boldsymbol{x}^\star) \end{bmatrix}, \quad \tilde{\boldsymbol{K}} \doteq \begin{bmatrix} \boldsymbol{K}_{AA} + \sigma_{\mathrm{n}}^2 \boldsymbol{I} & \boldsymbol{k}_{\boldsymbol{x}^\star, A} \\ \boldsymbol{k}_{\boldsymbol{x}^\star, A}^\top & k(\boldsymbol{x}^\star, \boldsymbol{x}^\star) \end{bmatrix}, \quad \boldsymbol{k}_{\boldsymbol{x}, A} \doteq \begin{bmatrix} k(\boldsymbol{x}, \boldsymbol{x}_1) \\ \vdots \\ k(\boldsymbol{x}, \boldsymbol{x}_n) \end{bmatrix}$$

- Deriving the conditional distribution, the Gaussian Process posterior is given by

Learning and Inference

- Denote the set of observed points by $A = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$
- Given: prior $f \sim \mathcal{GP}(\mu, k)$ and the noisy observations $y_i = f(\mathbf{x}_i) + \epsilon_i$ with $\epsilon_i \sim \mathcal{N}(0, \sigma_n^2)$
- The joint distribution of the observations $y_{1:n}$ and the noise free prediction f^* at a test point $\mathbf{x}^*$ can be written as:

$$\begin{bmatrix} \boldsymbol{y} \\ f^\star \end{bmatrix} \mid \boldsymbol{x}^\star, \boldsymbol{X} \sim \mathcal{N}(\tilde{\boldsymbol{\mu}}, \tilde{\boldsymbol{K}}), \quad \text{where} \qquad (4.$$

$$\tilde{\boldsymbol{\mu}} \doteq \begin{bmatrix} \boldsymbol{\mu}_A \\ \mu(\boldsymbol{x}^\star) \end{bmatrix}, \quad \tilde{\boldsymbol{K}} \doteq \begin{bmatrix} \boldsymbol{K}_{AA} + \sigma_{\mathrm{n}}^2 \boldsymbol{I} & \boldsymbol{k}_{\boldsymbol{x}^\star, A} \\ \boldsymbol{k}_{\boldsymbol{x}^\star, A}^\top & k(\boldsymbol{x}^\star, \boldsymbol{x}^\star) \end{bmatrix}, \quad \boldsymbol{k}_{\boldsymbol{x}, A} \doteq \begin{bmatrix} k(\boldsymbol{x}, \boldsymbol{x}_1) \\ \vdots \\ k(\boldsymbol{x}, \boldsymbol{x}_n) \end{bmatrix}$$

$\mu(\cdot) = 0$

- Deriving the conditional distribution, the Gaussian Process posterior is given by

$$f \mid \boldsymbol{x}_{1:n}, \boldsymbol{y}_{1:n} \sim \mathcal{GP}(\mu', k'), \quad \text{where}$$
$$\mu'(\boldsymbol{x}) \doteq \mu(\boldsymbol{x}) + \boldsymbol{k}_{\boldsymbol{x}, A}^\top (\boldsymbol{K}_{AA} + \sigma_{\mathrm{n}}^2 \boldsymbol{I})^{-1} (\boldsymbol{y}_A - \boldsymbol{\mu}_A).$$
$$k'(\boldsymbol{x}, \boldsymbol{x}') \doteq k(\boldsymbol{x}, \boldsymbol{x}') - \boldsymbol{k}_{\boldsymbol{x}, A}^\top (\boldsymbol{K}_{AA} + \sigma_{\mathrm{n}}^2 \boldsymbol{I})^{-1} \boldsymbol{k}_{\boldsymbol{x}', A}.$$

$= k_{xA}^T (K_{AA} + \sigma_n^2 \mathbb{1})^{-1} y_A$

- The predictive posterior at $\mathbf{x}^*$ is then given as:

Learning and Inference

- Denote the set of observed points by $A = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$
- Given: prior $f \sim \mathcal{GP}(\mu, k)$ and the noisy observations $y_i = f(\mathbf{x}_i) + \epsilon_i$ with $\epsilon_i \sim \mathcal{N}(0, \sigma_n^2)$
- The joint distribution of the observations $y_{1:n}$ and the noise free prediction f^* at a test point $\mathbf{x}^*$ can be written as:

$$\begin{bmatrix} \boldsymbol{y} \\ f^\star \end{bmatrix} \mid \boldsymbol{x}^\star, \boldsymbol{X} \sim \mathcal{N}(\tilde{\boldsymbol{\mu}}, \tilde{\boldsymbol{K}}), \quad \text{where} \qquad (4.$$

$$\tilde{\boldsymbol{\mu}} \doteq \begin{bmatrix} \boldsymbol{\mu}_A \\ \mu(\boldsymbol{x}^\star) \end{bmatrix}, \quad \tilde{\boldsymbol{K}} \doteq \begin{bmatrix} \boldsymbol{K}_{AA} + \sigma_{\mathrm{n}}^2 \boldsymbol{I} & \boldsymbol{k}_{\boldsymbol{x}^\star, A} \\ \boldsymbol{k}_{\boldsymbol{x}^\star, A}^\top & k(\boldsymbol{x}^\star, \boldsymbol{x}^\star) \end{bmatrix}, \quad \boldsymbol{k}_{\boldsymbol{x}, A} \doteq \begin{bmatrix} k(\boldsymbol{x}, \boldsymbol{x}_1) \\ \vdots \\ k(\boldsymbol{x}, \boldsymbol{x}_n) \end{bmatrix}$$

- Deriving the conditional distribution, the Gaussian Process posterior is given by

$$f \mid \boldsymbol{x}_{1:n}, \boldsymbol{y}_{1:n} \sim \mathcal{GP}(\mu', k'), \quad \text{where}$$
$$\mu'(\boldsymbol{x}) \doteq \mu(\boldsymbol{x}) + \boldsymbol{k}_{\boldsymbol{x}, A}^\top (\boldsymbol{K}_{AA} + \sigma_{\mathrm{n}}^2 \boldsymbol{I})^{-1} (\boldsymbol{y}_A - \boldsymbol{\mu}_A).$$
$$k'(\boldsymbol{x}, \boldsymbol{x}') \doteq k(\boldsymbol{x}, \boldsymbol{x}') - \boldsymbol{k}_{\boldsymbol{x}, A}^\top (\boldsymbol{K}_{AA} + \sigma_{\mathrm{n}}^2 \boldsymbol{I})^{-1} \boldsymbol{k}_{\boldsymbol{x}', A}.$$

- The predictive posterior at $\mathbf{x}^*$ is then given as:

$$f^\star \mid \boldsymbol{x}^\star, \boldsymbol{x}_{1:n}, \boldsymbol{y}_{1:n} \sim \mathcal{N}(\mu'(\boldsymbol{x}^\star), k'(\boldsymbol{x}^\star, \boldsymbol{x}^\star)).$$

ETH zürich

Gaussian Processes Sampling

How do we Sample Functions from a Gaussian Process?

Examine two approaches:

- Sampling All Points at Once
- Forward Sampling

Sampling All Points at Once

- Assume we want to sample a discrete subset of points $\mathbf{f} = [f_1, \ldots, f_n]$ at positions $\mathbf{x}_1, \ldots, \mathbf{x}_n$
- Assuming that $\mathbf{f} \sim \mathcal{N}(\boldsymbol{\mu}, \mathbf{K})$, we can also write $\mathbf{f}$ as

$$\underline{\varepsilon} \sim \mathcal{N}(\underline{0}, \underline{\underline{1}}) \rightarrow \underline{\underline{K}}^{1/2} \underline{\varepsilon} \sim \mathcal{N}(\underline{0}, \underline{\underline{K}})$$

$$\mathbf{f} = \mathbf{K}^{1/2}\boldsymbol{\epsilon} + \boldsymbol{\mu} \sim \mathcal{N}(\underline{\mu}, \underline{\underline{K}})$$

where $\mathbf{K}^{1/2}$ is the matrix square root of $\mathbf{K}$ and $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbb{1})$

Sampling All Points at Once

- Assume we want to sample a discrete subset of points $\mathbf{f} = [f_1, \ldots, f_n]$ at positions $\mathbf{x}_1, \ldots, \mathbf{x}_n$
- Assuming that $\mathbf{f} \sim \mathcal{N}(\boldsymbol{\mu}, \mathbf{K})$, we can also write $\mathbf{f}$ as

$$\mathbf{f} = \mathbf{K}^{1/2}\boldsymbol{\epsilon} + \boldsymbol{\mu}$$

  where $\mathbf{K}^{1/2}$ is the matrix square root of $\mathbf{K}$ and $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbb{1})$
- The matrix square root $\mathbf{K}^{1/2}$ can be computed via Cholesky decomposition
  - Since $\mathbf{K}$ is a positive semi-definite matrix, it can be written as $\mathbf{K} = \mathbf{L}\mathbf{L}^T$ where $\mathbf{L}$ is a lower triangular matrix!
  - Then: $\mathbf{f} = \mathbf{L}\boldsymbol{\epsilon} + \boldsymbol{\mu}$

Sampling All Points at Once

- Assume we want to sample a discrete subset of points $\mathbf{f} = [f_1, \ldots, f_n]$ at positions $\mathbf{x}_1, \ldots, \mathbf{x}_n$
- Assuming that $\mathbf{f} \sim \mathcal{N}(\boldsymbol{\mu}, \mathbf{K})$, we can also write $\mathbf{f}$ as

$$\mathbf{f} = \mathbf{K}^{1/2}\boldsymbol{\epsilon} + \boldsymbol{\mu}$$

  where $\mathbf{K}^{1/2}$ is the matrix square root of $\mathbf{K}$ and $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbb{1})$
- The matrix square root $\mathbf{K}^{1/2}$ can be computed via Cholesky decomposition
  - Since $\mathbf{K}$ is a positive semi-definite matrix, it can be written as $\mathbf{K} = \mathbf{L}\mathbf{L}^T$ where $\mathbf{L}$ is a lower triangular matrix!
  - Then: $\mathbf{f} = \mathbf{L}\boldsymbol{\epsilon} + \boldsymbol{\mu}$
- Sanity check:

Sampling All Points at Once

- Assume we want to sample a discrete subset of points $\mathbf{f} = [f_1, \ldots, f_n]$ at positions $\mathbf{x}_1, \ldots, \mathbf{x}_n$
- Assuming that $\mathbf{f} \sim \mathcal{N}(\boldsymbol{\mu}, \mathbf{K})$, we can also write $\mathbf{f}$ as

$$\mathbf{f} = \mathbf{K}^{1/2}\boldsymbol{\epsilon} + \boldsymbol{\mu}$$

  where $\mathbf{K}^{1/2}$ is the matrix square root of $\mathbf{K}$ and $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbb{1})$
- The matrix square root $\mathbf{K}^{1/2}$ can be computed via Cholesky decomposition
  - Since $\mathbf{K}$ is a positive semi-definite matrix, it can be written as $\mathbf{K} = \mathbf{L}\mathbf{L}^T$ where $\mathbf{L}$ is a lower triangular matrix!
  - Then: $\mathbf{f} = \mathbf{L}\boldsymbol{\epsilon} + \boldsymbol{\mu}$
- Sanity check:
  - $\mathbb{E}[\mathbf{f}] = \boldsymbol{\mu} + \mathbf{L}\mathbb{E}[\boldsymbol{\epsilon}] = \boldsymbol{\mu}$
  - $\mathrm{Var}[\mathbf{f}] = \mathbf{L}\mathrm{Var}[\boldsymbol{\epsilon}]\mathbf{L}^T = \mathbf{L}\mathbf{L}^T = \mathbf{K}$
- Problem: Computing the Cholesky decomposition takes $\mathcal{O}(n^3)$ time!

Forward Sampling

- Assume we want to sample a discrete subset of points $\mathbf{f} = [f_1, \ldots, f_n]$ at positions $\mathbf{x}_1, \ldots, \mathbf{x}_n$
- One can factorize the joint distribution as follows:
  - $p(f_1, \ldots, f_n) = \prod_{i=1}^{n} p(f_i \mid f_{1:i-1})$
- We can therefore obtain samples one-by-one, each time conditioning on one more observation:
  - $f_1 \sim p(f_1)$
  - $f_2 \sim p(f_2 \mid f_1)$
  - $f_3 \sim p(f_3 \mid f_1, f_2)$
  - …
- Problem: This approach also takes $\mathcal{O}(n^3)$ time!

How can we overcome the $\mathcal{O}(n^3)$ computational bottleneck?

- Will be covered in the next lecture and the next turorial!
- There will be a homework exercise on sparse Gaussian Processes!

ETH zürich

Old Exam Questions

Exam 2026

Suppose we are given a dataset $\mathcal{D} = \{(x_1, y_1), ..., (x_n, y_n)\}$ of n observations. We fit Gaussian processes with zero mean and the following five kernels to the data $\mathcal{D}$.

- $k_1(x, x') = (1 + xx')^5$
- $k_2(x, x') = \cos\left(3\,|x - x'|\right)$
- $k_3(x, x') = \exp\left(-2\,(x - x')^2\right)$
- $k_4(x, x') = \exp\left(-2\,|x - x'|\right)$
- $k_5(x, x') = \left(1 + 2\sqrt{3}\,|x - x'|\right) \cdot \exp\left(-2\sqrt{3}\,|x - x'|\right)$

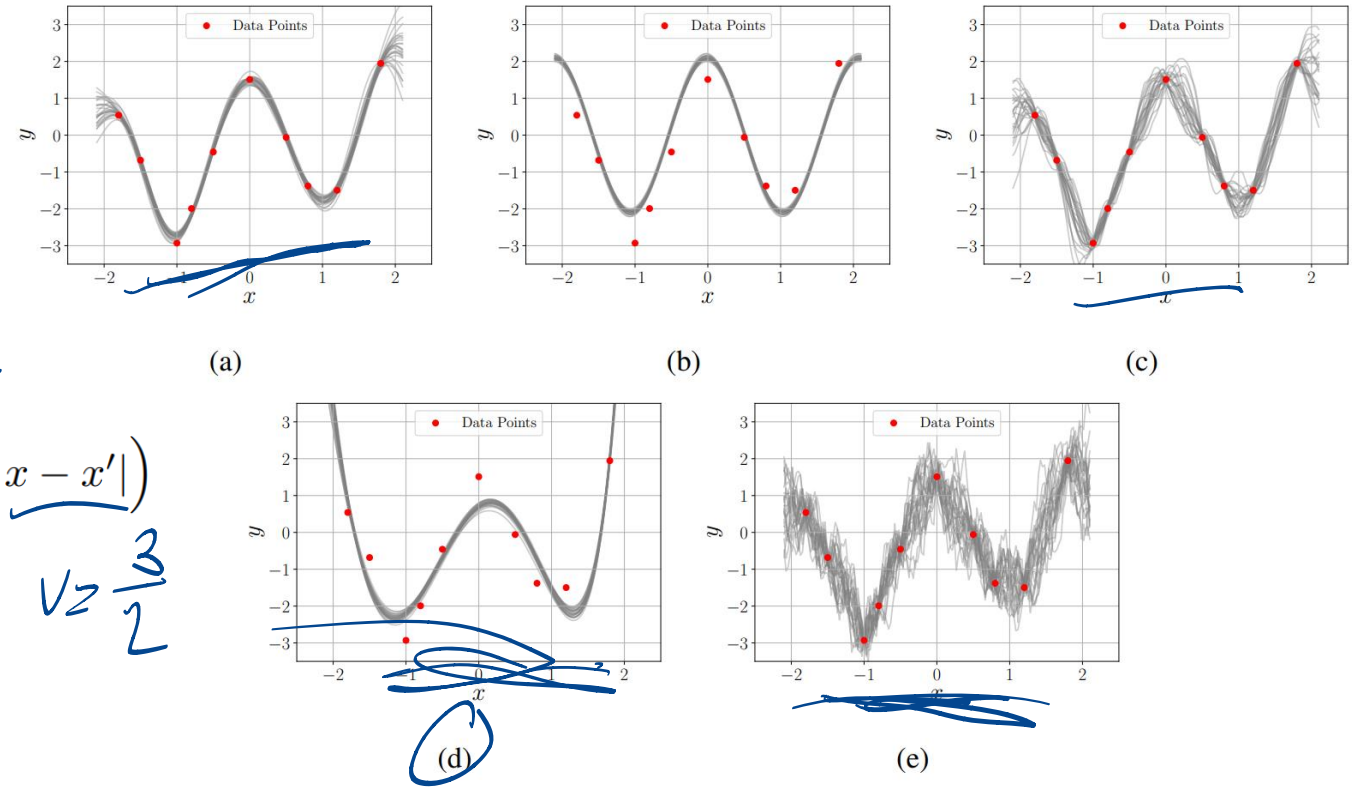


Figure 1: Samples of posterior Gaussian processes for different kernels k. The red dots represent the data $\mathcal{D}$.

There is a one-to-one correspondence between the kernel functions $\{k_1, ..., k_5\}$ and the plots in Fig. 1.

Exam 2024

$(x_1, y_1) = (0, 1)$ $(x_2, y_2) = (1, 0)$ $k(x, x') = xx'$

Which of the following values for σ^2 maximizes $\log p(y_1, y_2 | x_1, x_2, \sigma^2)$?

A $\sigma^2 = \frac{1}{2}$ B $\sigma^2 = \frac{1}{\sqrt{2}}$ C $\sigma^2 = 1$ D $\sigma^2 = \sqrt{2}$ E $\sigma^2 = 2$

Exam 2025

Isotropy ⇒ Stationarity

$k(x, x') = \tilde{k}(x - x')$

↳ isotropic

Question 10 *(1 point)* Every stationary kernel $k : \mathbb{R}^n \times \mathbb{R}^n \to \mathbb{R}$ satisfies $k(\boldsymbol{x}, \boldsymbol{x}') = \tilde{k}(\|\boldsymbol{x} - \boldsymbol{x}'\|_2)$ for some function $\tilde{k} : \mathbb{R} \to \mathbb{R}$.

A True

B False

$\mathbb{R}$

Isotropy ⇔ Stationarity

Exam 2024

g symmetric
g PSD

The function $g(x, x') = k(ax, x')$ must be a valid covariance function, assuming $k(x, x')$ is a covariance function and $a > 0$ is a constant.

A True B False

$k(x, x') = x^T x$

$g(x, x') = a\, x^T x$

Exam 2024

Consider an undirected square graph, i.e. the graph $G = (V, E)$ with vertices $V = \{1, 2, 3, 4\}$ and edges $E = \{\{1, 2\}, \{2, 3\}, \{3, 4\}, \{4, 1\}\}$, as depicted below:

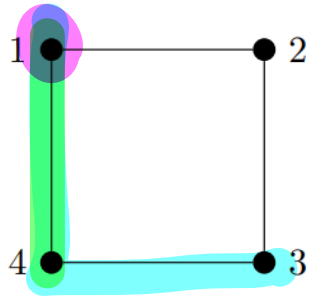


Figure 1: Corresponding to Question 11.

Define the kernel $k_l : V \times V \to \mathbb{R}$ on the vertices of G similarly to the standard squared exponential kernel, by $k_l(x, x') = \exp(-\frac{d(x,x')^2}{2l^2})$, where $d(x, x')$ is the length of the shortest path on G between x and x' and $l \neq 0$. The distance of a vertex from itself, i.e. $d(x, x)$, is zero. **Mark all that apply.**

- [A] For all permissible values of l, k_l is a symmetric function, i.e. $k_l(x, x') = k_l(x', x)$.
- [B] k_l for $l = 1$ is a valid kernel, i.e. a symmetric positive semidefinite function.
- [C] For all permissible values of l, k_l is a valid kernel, i.e. a symmetric positive semidefinite function.

Hint: $(1, -1, 1, -1)^\top$ is an eigenvector of the matrix $\mathbf{K} = \begin{pmatrix} k_l(1,1) & k_l(1,2) & k_l(1,3) & k_l(1,4) \\ k_l(2,1) & k_l(2,2) & k_l(2,3) & k_l(2,4) \\ k_l(3,1) & k_l(3,2) & k_l(3,3) & k_l(3,4) \\ k_l(4,1) & k_l(4,2) & k_l(4,3) & k_l(4,4) \end{pmatrix}$.

$Kv = (a - 2b + c)\begin{pmatrix} 1 \\ -1 \\ 1 \\ -1 \end{pmatrix}$

y $-0.08 < 0$

$= \begin{pmatrix} a & b & c & b \\ b & a & b & c \\ c & b & a & b \\ b & c & b & a \end{pmatrix}$

Exam 2024

For this section, we consider the data-generating process of $x_i \in \mathbb{R}$ and $y_i \in \mathbb{R}$ given by

$$y_i = f(x_i) + \epsilon_i, \qquad \epsilon_i \sim \mathcal{N}(0, \sigma^2),$$

where f is an unknown function and $\sigma > 0$ a known scalar. We model prior of the function f by a Gaussian process $f \sim \mathrm{GP}(0, k)$ with covariance function k.

Suppose we observe two data points $(x_1, y_1) = (0, 1)$ and $(x_2, y_2) = (1, 0)$ in the setting described at the top of this section. We now use the linear kernel

$$k(x, x') = xx'$$

to model the unknown function f. To tune the variance hyperparameter $\sigma^2 > 0$, we maximize the marginal likelihood $p(y_1, y_2 | x_1, x_2, \sigma^2)$. Which of the following values for σ^2 maximizes $\log p(y_1, y_2 | x_1, x_2, \sigma^2)$? You may assume that any $\sigma^2 > 0$ satisfying $\frac{d}{d\sigma^2} \log p(y_1, y_2 | x_1, x_2, \sigma^2) = 0$ is a global maximum.

Hint: It holds that $\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc$.

A $\sigma^2 = \frac{1}{2}$ B $\sigma^2 = \frac{1}{\sqrt{2}}$ C $\sigma^2 = 1$ D $\sigma^2 = \sqrt{2}$ E $\sigma^2 = 2$